CS492D: Diffusion Models and Their Applications

Classifier-Free Guidance / Latent Diffusion / ControlNet / LoRA

LECTURE 7
MINHYUK SUNG

Fall 2024
KAIST

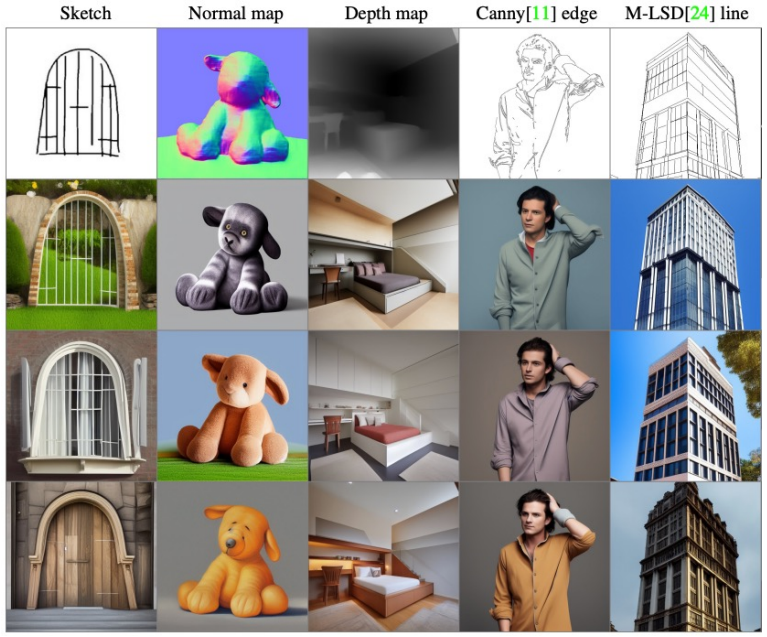
Guidance Using Additional Information

If we have additional information about the data, such as class labels for images, can we utilize this information to improve the quality of generated outputs?

Ho and Salimans, Classifier-Free Diffusion Guidance, NeurIPS 2021 Workshop.

Fine-Tuning-Based Methods

Can we fine-tune a pretrained diffusion model for conditional generation or personalization?



Zhang et al., Adding Conditional Control to Text-to-Image Diffusion Models, ICCV 2022.

Ruiz et al., DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation, CVPR 2023.

Content

1. Classifier Guidance / Classifier-Free Guidance

2. Fine-Tuning-Based Methods

    1. ControlNet

    2. LoRA

Classifier Guidance / Classifier-Free Guidance (CFG)

The CIFAR-10 dataset. https://www.cs.toronto.edu/~kriz/cifar.html

Guidance Using Additional Information

How to use class labels or some additional information about the data to improve the quality of generated outputs?

Classifier Guidance

Dhariwal and Nichol, Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021.

Classifier Guidance

Assume that data $\boldsymbol{x}$ and class label y pairs are given:

$$p(\boldsymbol{x}_t, y) = p(\mathbf{x}_t)p(y|\mathbf{x}_t)$$

( , dog)

Classifier Guidance

At each timestep t, we are interested in the score function $\nabla_{\boldsymbol{x}_t} \log p(\boldsymbol{x}_t, y)$:

$$\nabla_{\boldsymbol{x}_t} \log p(\boldsymbol{x}_t, y) = \nabla_{\boldsymbol{x}_t} \log p(\mathbf{x}_t) + \nabla_{\boldsymbol{x}_t} \log p(y|\mathbf{x}_t)$$

$$= -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\varepsilon}_\theta(\boldsymbol{x}_t, t) + \nabla_{\boldsymbol{x}_t} \log p(y|\mathbf{x}_t)$$

New noise!

$$= -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \left(\boldsymbol{\varepsilon}_\theta(\mathbf{x}_t, t) - \sqrt{1-\bar{\alpha}_t} \nabla_{\boldsymbol{x}_t} \log p(y|\boldsymbol{x}_t) \right)$$

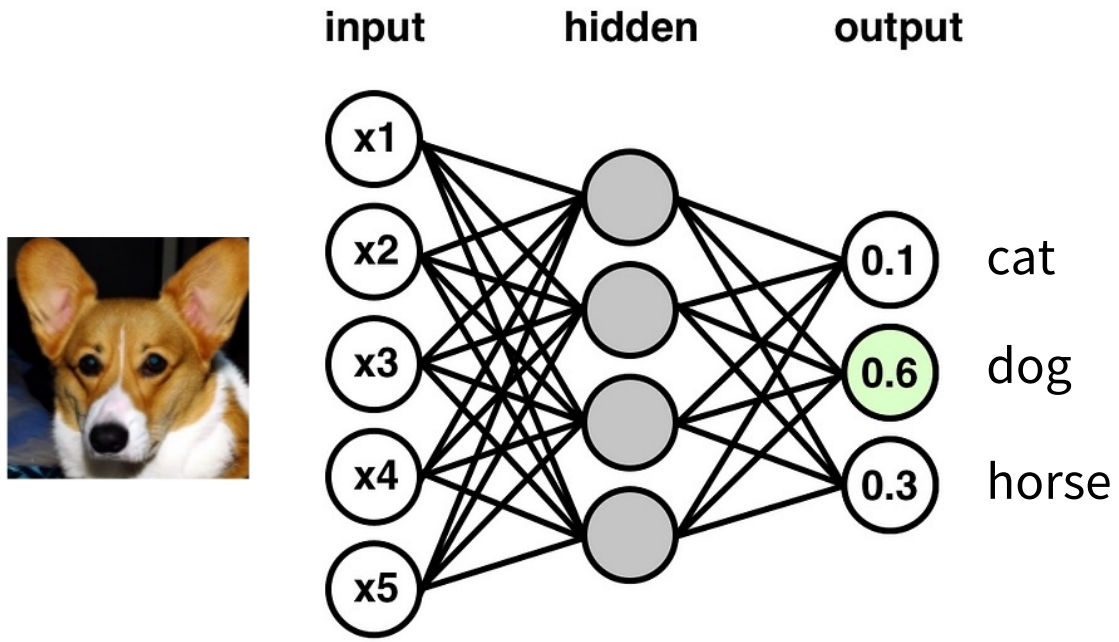
How to compute this?

https://towardsdatascience.com/coding-up-a-neural-network-classifier-from-scratch-977d235d8a24

Classifier Guidance

How to compute $\nabla_{\boldsymbol{x}_t} \log p(y|\boldsymbol{x}_t)$?

How to learn $p(y|\boldsymbol{x}_t)$? Train a classification network!



Classifier Guidance

How to compute $\nabla_{\boldsymbol{x}_t} \log p(y|\boldsymbol{x}_t)$?

- Train a classifier $p_\phi(y|\boldsymbol{x}_t)$, taking noisy data $\boldsymbol{x}_t$ (and time step t) as input and classifying it.

- Update the noise prediction $\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, t)$ as follows:

$$\tilde{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) = \hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, t) - \sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t} \log p_\phi(y|\boldsymbol{x}_t)$$

Condition Enhancement

The strength of the classifier guidance can be controlled by adding a weight $\omega \geq 1$:

$$\tilde{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) = \hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, t) - \omega\sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t}\log p_\phi(y|\boldsymbol{x}_t)$$

Classifier Guidance

$$\tilde{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) = \hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, t) - \omega\sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t}\log p_\phi(y|\boldsymbol{x}_t)$$

$$= -\sqrt{1-\bar{\alpha}_t}\big(\nabla_{\boldsymbol{x}_t}\log p_\phi(\boldsymbol{x}_t) + \omega\nabla_{\boldsymbol{x}_t}\log p_\phi(y|\boldsymbol{x}_t)\big)$$

$$= -\sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t}\log p_\phi(\boldsymbol{x}_t) + \nabla_{\boldsymbol{x}_t}\log p_\phi(y|\boldsymbol{x}_t)^\omega$$

$$= -\sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t}\log\big(p_\phi(\boldsymbol{x}_t)p_\phi(y|\boldsymbol{x}_t)^\omega\big)$$

Classifier Guidance

$\boxed{p_\phi(\boldsymbol{x}_t)p_\phi(y|\boldsymbol{x}_t)^\omega}$ instead of $p(\mathbf{x}_t)p(y|\mathbf{x}_t) = p(\boldsymbol{x}_t, y)$

$p_\phi(y|\boldsymbol{x}_t)^\omega$ amplifies large values.

$\Longrightarrow$ Focusing more on the modes of the classifer.

$\Longrightarrow$ Higher fidelity to the input label but less diversity.

Classifier Guidance

- (+) Not only is the generation quality improved, but **conditional generation** with labels also becomes possible!

- (−) Additional training of a classifier is required.

Classifier-Free Guidance

Ho and Salimans, Classifier-Free Diffusion Guidance, NeurIPS Workshop 2021.

Classifier-Free Guidance (CFG)

Jointly train the conditional and unconditional diffusion models by introducing a null label ϕ.

$\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t)$: Noise prediction *without* condition.

Condition Enhancement

How can we enhance the conditioning?

We can extrapolate the conditional noise $\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t)$ from the null-condition noise $\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t)$:

$$\tilde{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) = (1+\omega)\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) - \omega\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t)$$

where $\omega \geq 0$.

Connection to Classifier Guidance

Why is it an extrapolation?

Let $\lambda \coloneqq 1 + \omega \quad (\lambda \geq 1)$

$$\tilde{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) = \lambda \hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) + (1 - \lambda)\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t)$$

$$= \hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t) + \lambda\big(\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) - \hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t)\big)$$

Connection to Classifier Guidance

$$\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t) + \lambda\left(\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) - \hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t)\right)$$

$$= -\sqrt{1-\bar{\alpha}_t}\left(\nabla_{\boldsymbol{x}_t} \log p(\boldsymbol{x}_t) + \lambda\left(\nabla_{\boldsymbol{x}_t} \log p(\boldsymbol{x}_t|y) - \nabla_{\boldsymbol{x}_t} \log p(\boldsymbol{x}_t)\right)\right)$$

$$= -\sqrt{1-\bar{\alpha}_t}\nabla_{\boldsymbol{x}_t} \log\left(p(\boldsymbol{x}_t)\left(\frac{p(\boldsymbol{x}_t|y)}{p(\boldsymbol{x}_t)}\right)^\lambda\right)$$

Connection to Classifier Guidance

$$p(\boldsymbol{x}_t)\left(\frac{p(\boldsymbol{x}_t|y)}{p(\boldsymbol{x}_t)}\right)^{\lambda}$$

$$\propto p(\boldsymbol{x}_t)\left(\frac{p(\boldsymbol{x}_t|y)\cdot p(y)}{p(\boldsymbol{x}_t)}\right)^{\lambda} \qquad \text{Multiply constant } p(y)^{\lambda}$$

$$=\boxed{p(\boldsymbol{x}_t)\big(p(y|\boldsymbol{x}_t)\big)^{\lambda}} \qquad \text{Bayes' theorem}$$

Classifier Guidance

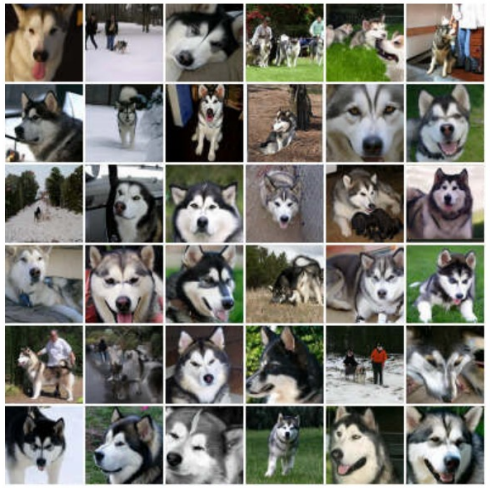
$p_\phi(\boldsymbol{x}_t)p_\phi(y|\boldsymbol{x}_t)^\omega$ instead of $p(\mathbf{x}_t)p(y|\mathbf{x}_t) = p(\boldsymbol{x}_t, y)$

It is equivalent to the condition enhancement in Classifier Guidance!

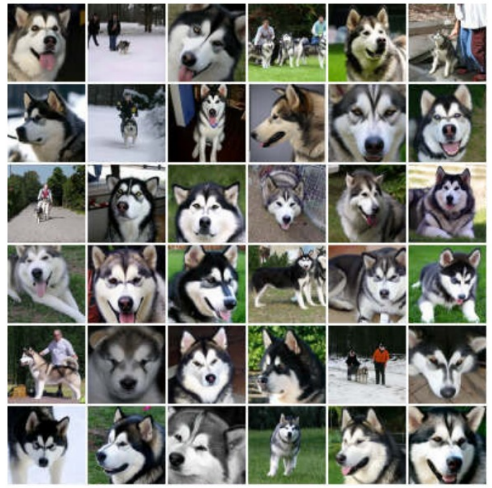
Meng et al., On Distillation of Guided Diffusion Models, CVPR 2023.

Classifier-Free Guidance (CFG)

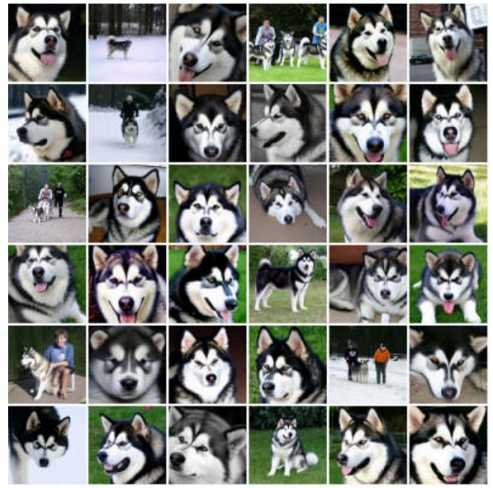
(a) $w = 0$



(b) $w = 1$



(c) $w = 2$



(d) $w = 4$

Classifier-Free Guidance (CFG)

Model	FID (↓)
BigGAN-deep, max IS (Brock et al., 2019)	25
BigGAN-deep (Brock et al., 2019)	5.7
CDM (Ho et al., 2021)	3.52
LOGAN (Wu et al., 2019)	3.36
Classifier Guidance: ADM-G (Dhariwal & Nichol, 2021)	2.97
Ours	$T = 128$ / 256 / 1024
$w = 0.0$	8.11 / 7.27 / 7.22
$w = 0.1$	5.31 / 4.53 / 4.5
$w = 0.2$	3.7 / 3.03 / 3
$w = 0.3$	3.04 / **2.43** / **2.43**
$w = 0.4$	3.02 / 2.49 / 2.48
$w = 0.5$	3.43 / 2.98 / 2.96
$w = 0.6$	4.09 / 3.76 / 3.73
$w = 0.7$	4.96 / 4.67 / 4.69
$w = 0.8$	5.93 / 5.74 / 5.71
$w = 0.9$	6.89 / 6.8 / 6.81
$w = 1.0$	7.88 / 7.86 / 7.8
$w = 2.0$	15.9 / 15.93 / 15.75
$w = 3.0$	19.77 / 19.77 / 19.56
$w = 4.0$	21.55 / 21.53 / 21.45

$T = 256$

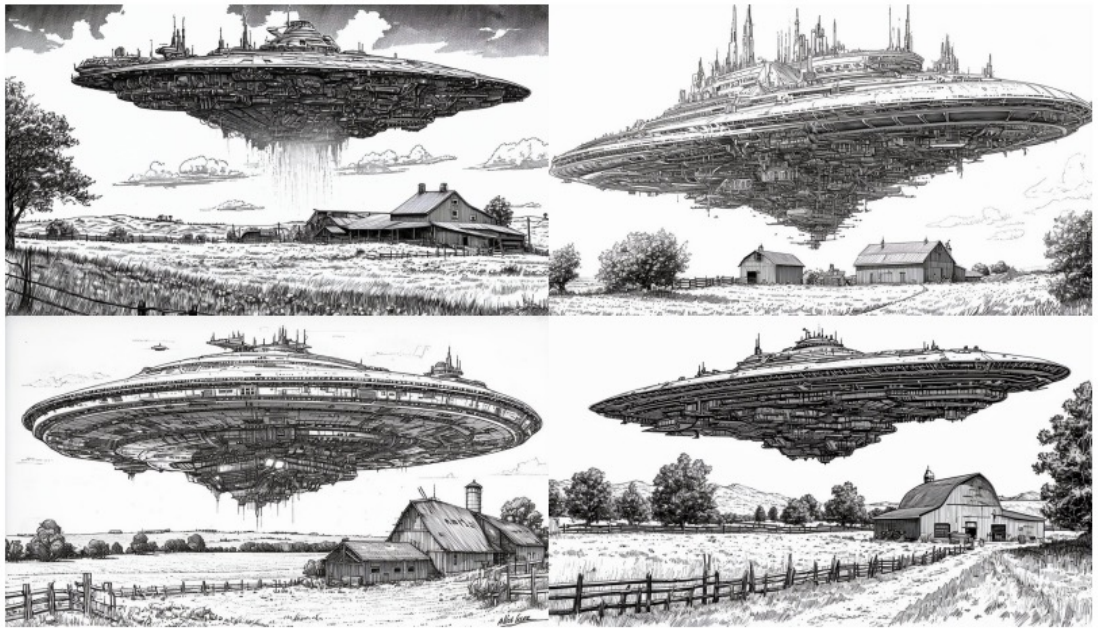
ImageNet Results

Classifier-Free Guidance (CFG)

- (+) Very easy to implement.
- (+) Versatile; can be used not only for labels but also for any additional information (e.g., text descriptions).
- (−) The noise predictor needs to be evaluated twice in the generation process.

Stable Diffusion 3. Esser et al., Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. ICML 2024.

Example: Text-to-Image (T2I)



detailed pen and ink drawing of a massive complex alien space ship above a farm in the middle of nowhere.

photo of a bear wearing a suit and tophat in a river in the middle of a forest holding a sign that says "I cant bear it".

Stable Diffusion 3. Esser et al., Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. ICML 2024.

Example: Text-to-Image (T2I)

tilt shift aerial photo of a cute city made of sushi on a wooden table in the evening.

dark high contrast render of a psychedelic tree of life illuminating dust in a mystical cave.

Stable Diffusion 3. Esser et al., Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. ICML 2024.

Example: Text-to-Image (T2I)

an anthropomorphic fractal person behind the counter at a fractal themed restaurant.

beautiful oil painting of a steamboat in a river in the afternoon. On the side of the river is a large brick building with a sign on top that says ŠD3.

Stable Diffusion 3. Esser et al., Scaling Rectified Flow Transformers for High-Resolution Image Synthesis. ICML 2024.

Example: Text-to-Image (T2I)

an anthopomorphic pink donut with a mustache and cowboy hat standing by a log cabin in a forest with an old 1970s orange truck in the driveway

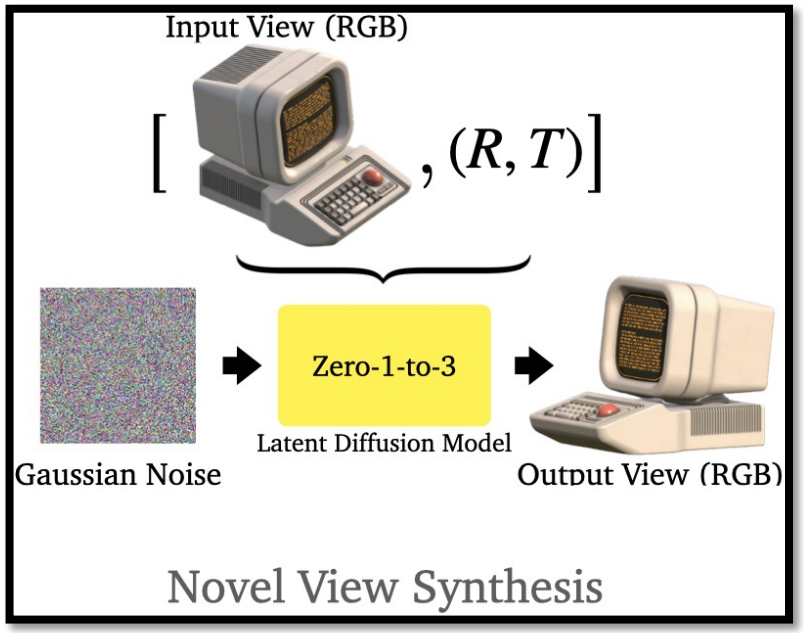
fox sitting in front of a computer in a messy room at night. On the screen is a 3d modeling program with a line render of a zebra.

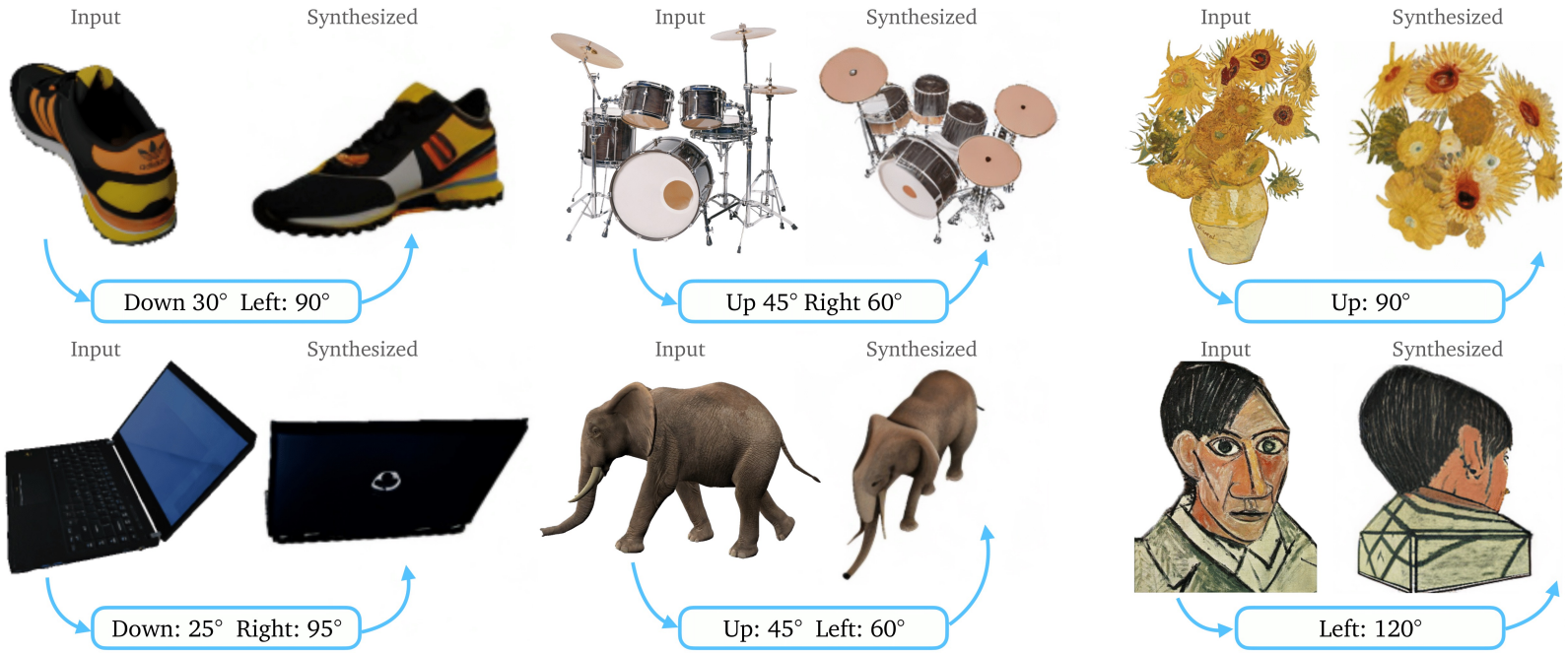
Brooks et al., InstructPix2Pix Learning to Follow Image Editing Instructions, CVPR 2023.

Example: Image-to-Image

Liu et al., Zero-1-to-3: Zero-shot One Image to 3D Object, ICCV 2023.

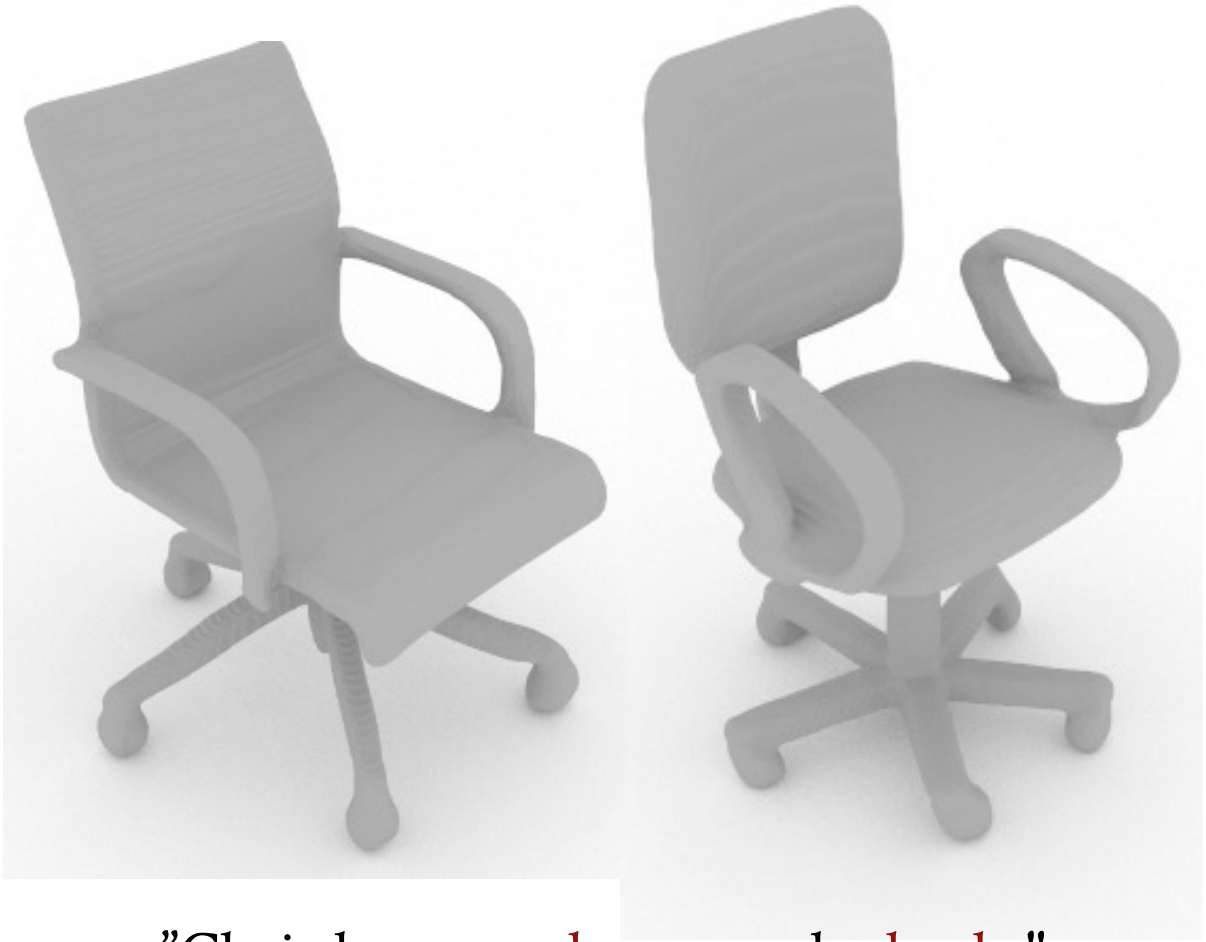
Example: Zero 1-to-3





Koo et al., SALAD: Part-Level Latent Diffusion for 3D Shape Generation and Manipulation, ICCV 2023.

Example: Text-to-3D



”Chair has round arms and wheels."

”Its the one with gaps in the back."

Lee et al., InterHandGen: Two-Hand Interaction Generation via Cascaded Reverse Diffusion, CVPR 2024.

Example: Hand-to-Hand

One hand is generated based on the condition of the other hand.

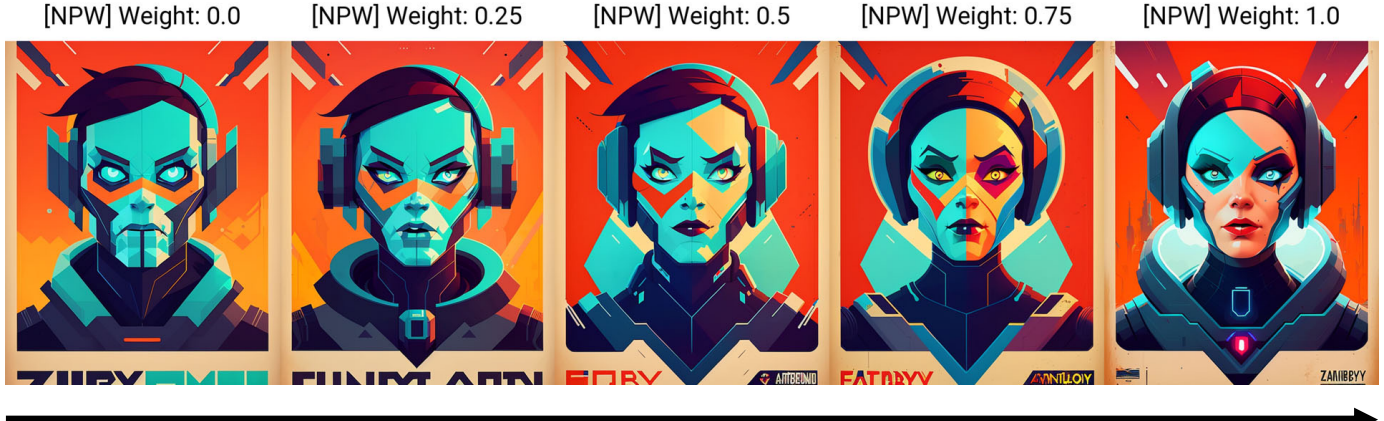
Credit: Duygu Ceylan

Audio-to-Image

https://github.com/muerrilla/stable-diffusion-NPW?tab=readme-ov-file

Negative Prompt

Prompt: Portrait of zimby anton fadeev cyborg propaganda poster



Negative Prompt

$$\tilde{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) = (1+\omega)\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) - \omega\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, \phi, t)$$

$$\tilde{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y, t) = (1+\omega)\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y_+, t) - \omega\hat{\boldsymbol{\varepsilon}}_\theta(\boldsymbol{x}_t, y_-, t)$$

Positive prompt

Negative prompt

Latent Diffusion

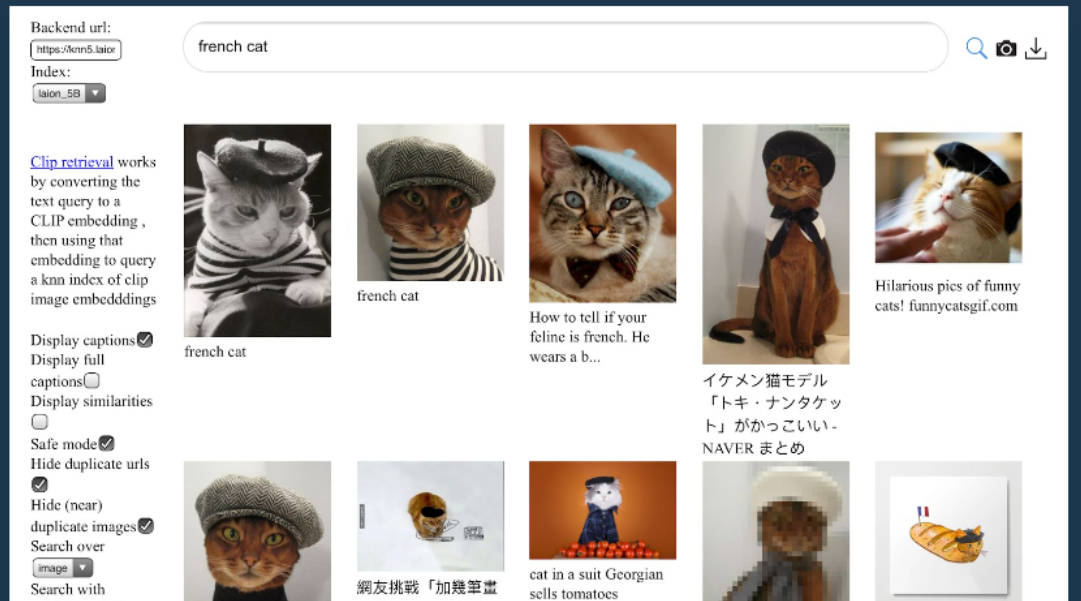
Rombach et al., High-Resolution Image Synthesis with Latent Diffusion Models, CVPR 2022.

LAION-5B: A NEW ERA OF OPEN LARGE-SCALE MULTI-MODAL DATASETS

by: Romain Beaumont, 31 Mar, 2022

We present a dataset o 5,85 billion CLIP-filtered image-text pairs, 14x bigger than LAION-400M, previously the biggest openly accessible image-text dataset in the world - see also our NeurIPS2022 paper

Authors: Christoph Schuhmann, Richard Vencu, Romain Beaumont, Theo Coombes, Cade Gordon, Aarush Katta, Robert Kaczmarczyk, Jenia Jitsev



Rombach et al., High-Resolution Image Synthesis with Latent Diffusion Models, CVPR 2022.

High-Resolution Images

Rombach et al., High-Resolution Image Synthesis with Latent Diffusion Models, CVPR 2022.
Dhariwal and Nichol, Diffusion Models Beat GANs on Image Synthesis, NeurIPS 2021.

High-Resolution Images

- Training: Hundreds of GPU days (e.g. 150 - 1000 V100 days)
- Generation: Generating 50k samples takes approximately 5 days on a single A100 GPU.

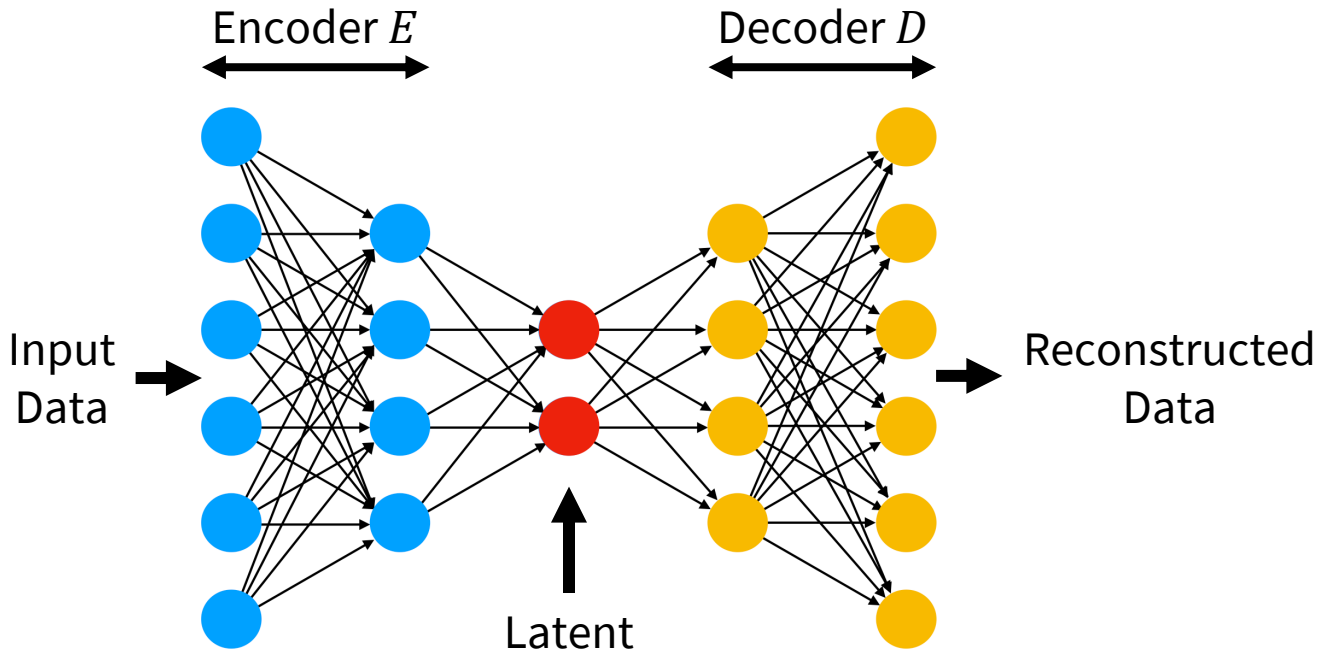
How to reduce the computation time?

→ Image compression.

Image Compression

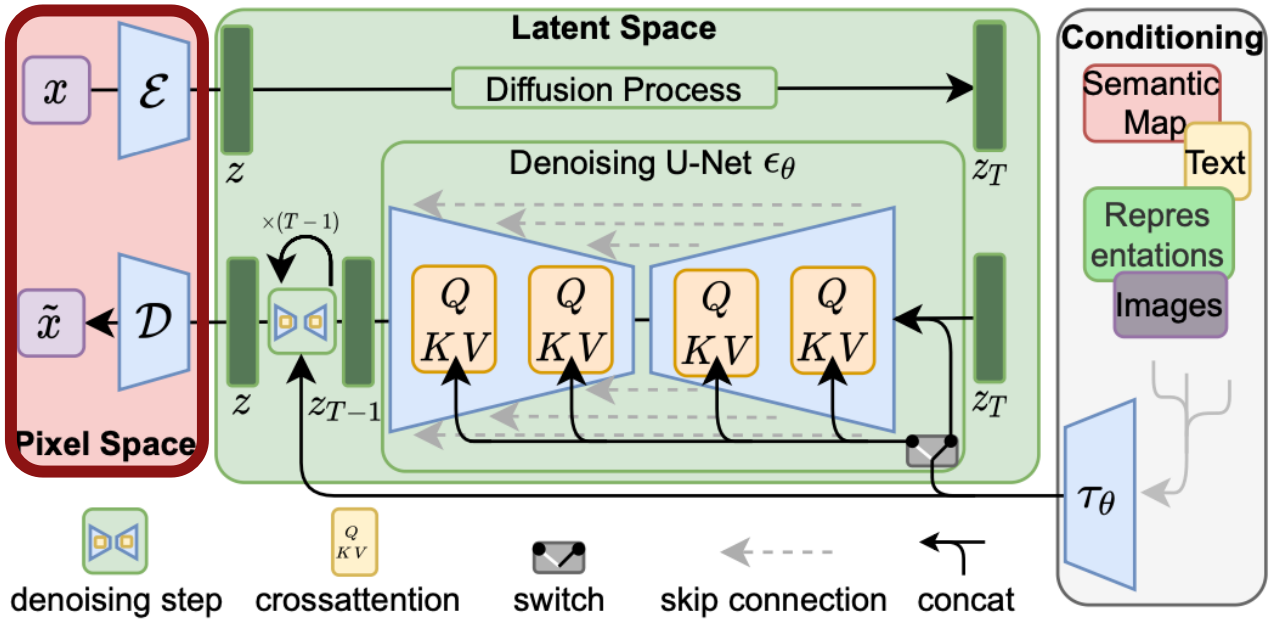
Pretrain an autoencoder (with KL-divergence regularization loss) to decrease the resolution from, for example,

$$512\times512\times3 \Longrightarrow 64\times64\times4.$$



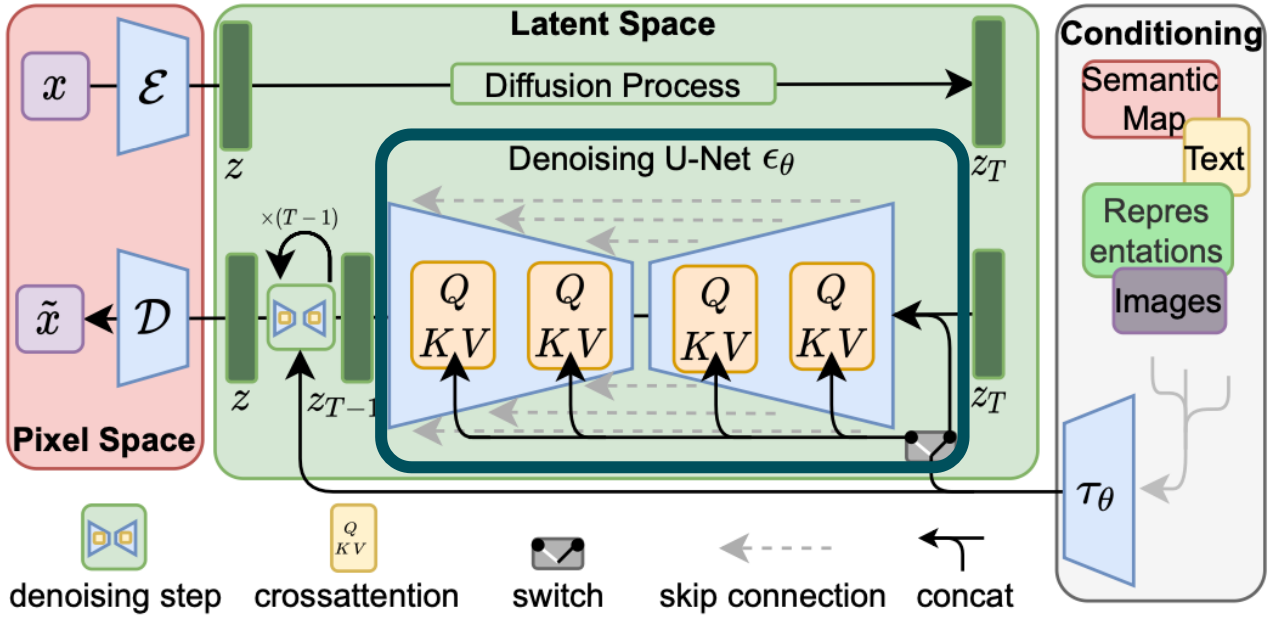
Latent Diffusion

- Image encoder/decoder are pretrained.



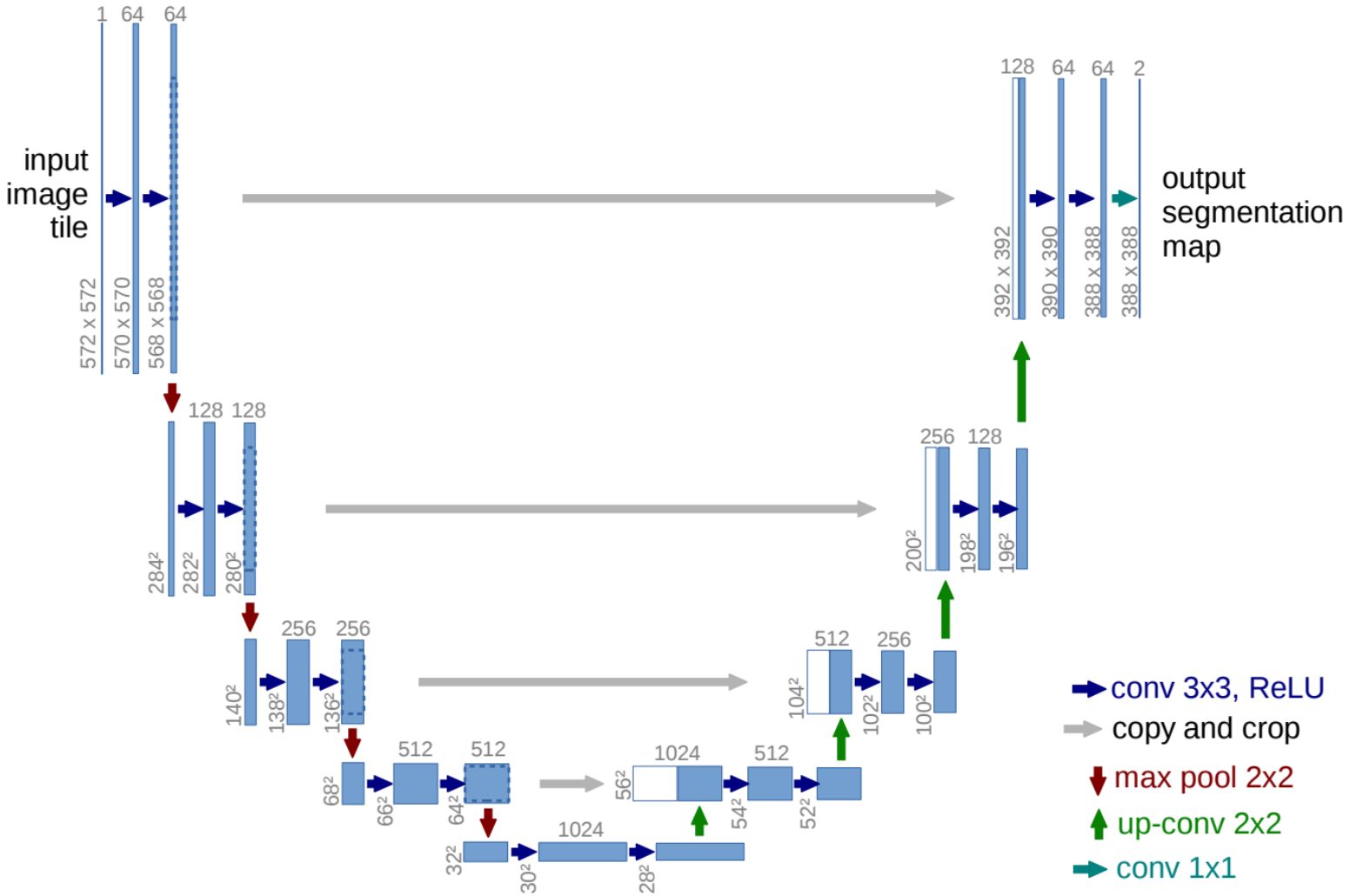
Latent Diffusion

- The noise predictor is trained in the latent space.
- For noise prediction, earlier models use U-Net with attention module at each layer.



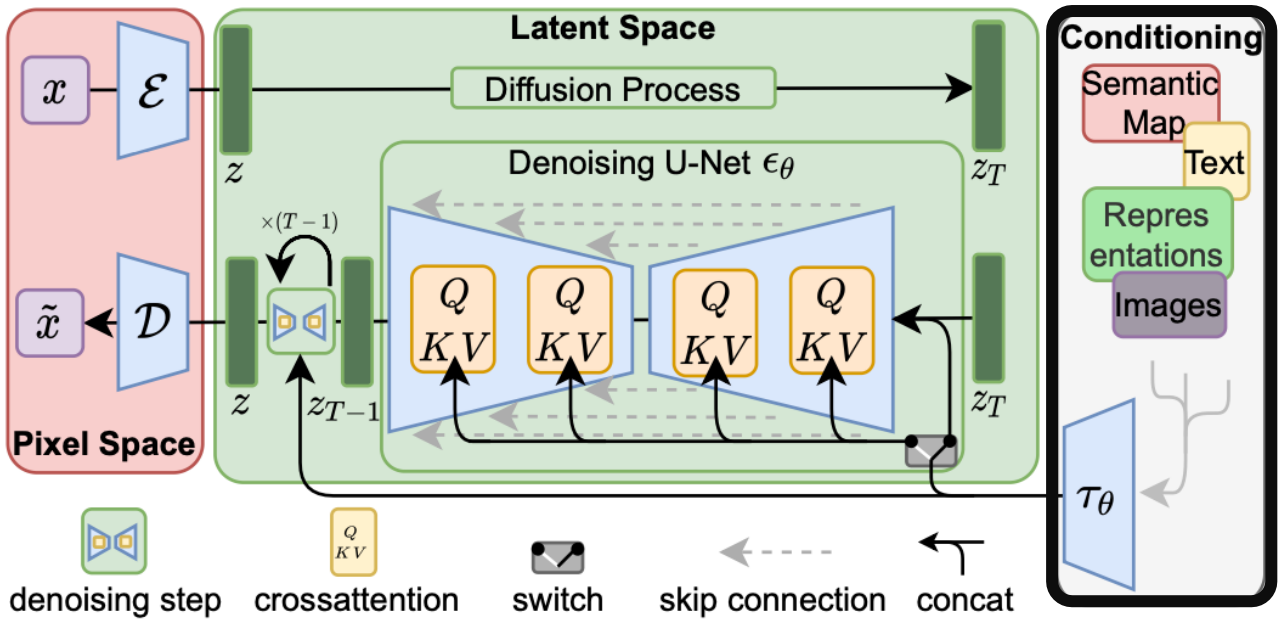
Ronneberger et al., U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015.

Latent Diffusion



Latent Diffusion

- The input condition is incorporated into the cross-attention module.



Latent Diffusion

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) \cdot V$$

- Query Q: The output of each U-Net layer.
- Key K and Value V: The outputs from the input condition encoder.

Guest Lecture 1

The Power of Attention Layers in Generative Models

October 10, 2024 (Thu)
4:00 pm KST

Or Patashnik

PhD student at Tel-Aviv University

https://github.com/huggingface/diffusers
https://huggingface.co/docs/diffusers/en/index

Diffusers

D🧨ffusers

🤗 Diffusers is the go-to library for state-of-the-art pretrained diffusion models for generating images, audio, and even 3D structures of molecules. Whether you're looking for a simple inference solution or training your own diffusion models, 🤗 Diffusers is a modular toolbox that supports both. Our library is designed with a focus on usability over performance, simple over easy, and customizability over abstractions.

🤗 Diffusers offers three core components:

- State-of-the-art diffusion pipelines that can be run in inference with just a few lines of code.
- Interchangeable noise schedulers for different diffusion speeds and output quality.
- Pretrained models that can be used as building blocks, and combined with schedulers, for creating your own end-to-end diffusion systems.

Latent Diffusion

Method	FID↓	IS↑	Precision↑	Recall↑	Nparams
BigGan-deep [3]	6.95	203.6±2.6	**0.87**	0.28	340M
ADM [15]	10.94	100.98	0.69	**0.63**	554M
ADM-G [15]	4.59	186.7	0.82	0.52	608M
LDM-4 (ours)	10.56	103.49±1.24	0.71	0.62	400M
LDM-4-G (ours)	**3.60**	**247.67**±5.59	**0.87**	0.48	400M

Fréchet Inception Distance (FID)

Heusel et al., GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium, NeurIPS 2017.

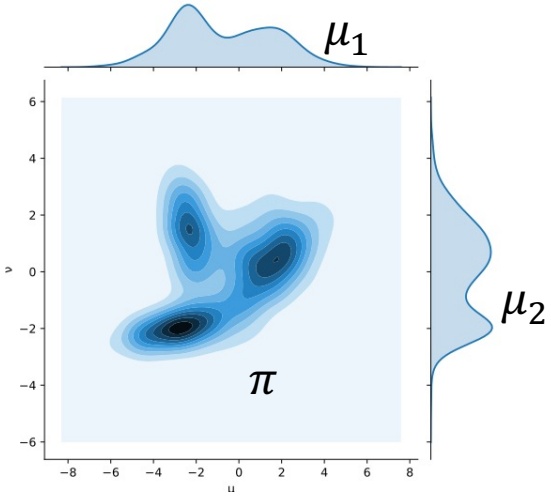
Image credit: Wikipedia

p-Wasserstein Distance

a.k.a (in a discrete setup)
Fréchet distance / Earth movers' distance

$$W_p(\mu,\nu) \coloneqq \inf_{\pi\in\Gamma(\mu,\nu)} \left(\iint_{X\times X} d(x,y)^p \,\mathrm{d}\pi(x,y) \right)^{1/p}$$

- μ and ν: Probability distributions
- $\Gamma(\mu,\nu)$: A set of possible joint distributions

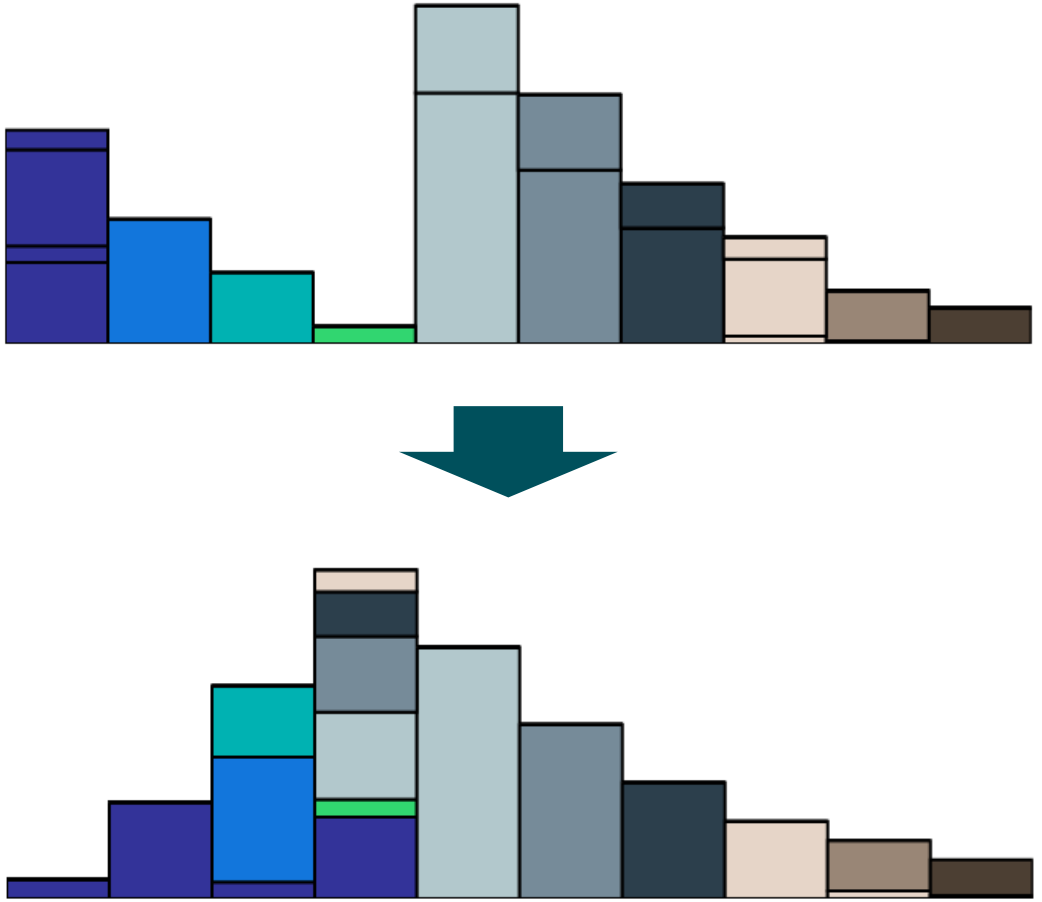


https://www.oreilly.com/library/view/hands-on-generative-adversarial/9781789530513/bd9f0bcb-800e-45c2-bc34-f93d83f0a763.xhtml

p-Wasserstein Distance

A discrete setup:

The minimum amount of the sum of mass×distance to match one distribution to the other.



p-Wasserstein Distance

When the two distributions are normal distributions:

$$\mu = \mathcal{N}(m_1, C_1) \text{ and } \nu = \mathcal{N}(m_2, C_1),$$

2-Wasserstein distance becomes the following:

$$W(\mu, \nu) = \|m_1 - m_2\|_2^2 + \mathrm{Tr}\left(C_1 + C_2 - 2\left(C_2^{1/2} C_1 C_2^{1/2}\right)^{1/2}\right).$$

[1] Szegedy et al., Rethinking the Inception Architecture for Computer Vision, CVPR 2016.

Fréchet Inception Distance (FID)

Compare the distributions of the real images and generated images in the latent space of Inception v3[1] network.

- Map all real and generated images to the latent space of the Inception v3 network.
- Model each distribution as a normal distribution and compute the mean and covariance.
- Compute the 2-Wasserstein (Fréchet) distance.

Parmar et al., On Aliased Resizing and Surprising Subtleties in GAN Evaluation, CVPR 2022.

Fréchet Inception Distance (FID)

Is FID a perfect metric for generative models?

ControlNet: Few-Shot Adaptation

Zhang et al., Adding Conditional Control to Text-to-Image Diffusion Models, ICCV 2022.

ControlNet [Zhang et al., 2023]

Zhang et al., Adding Conditional Control to Text-to-Image Diffusion Models, ICCV 2022.

ControlNet [Zhang et al., 2023]

Should we have 5 billion input-output pairs to train conditional image diffusion models?

ControlNet [Zhang et al., 2023]

Can we convert a pretrained unconditional image diffusion model into an image-conditioned generative model using a relatively much smaller set of input-output pairs ($\sim 100k$)?

Zhang et al., Adding Conditional Control to Text-to-Image Diffusion Models, ICCV 2022.

ControlNet [Zhang et al., 2023]

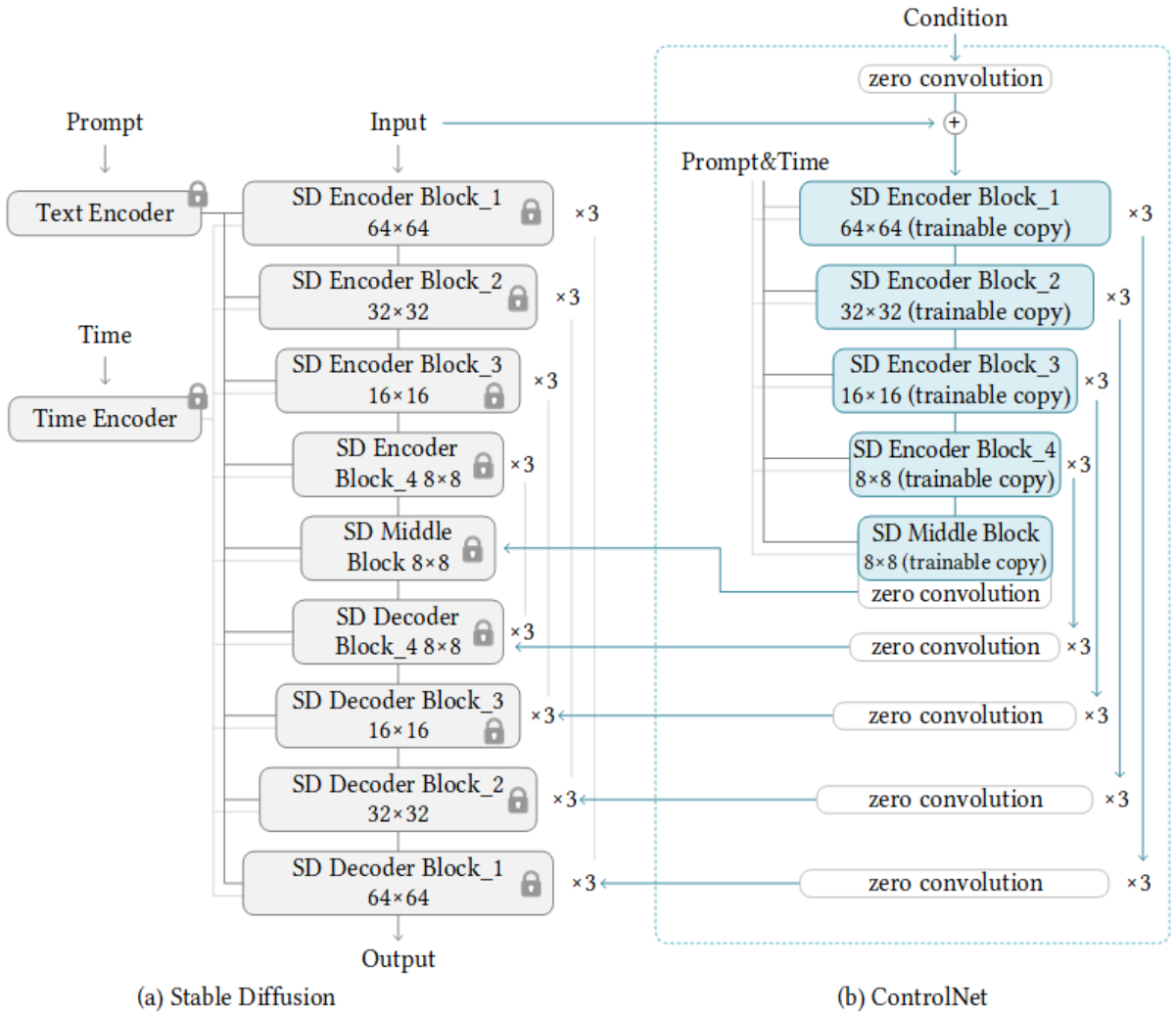
Let's consider a special case where both the input condition and the output are images.

Fine-Tuning Approaches

Fine-tune the pretrained noise prediction network using only

- a small dataset, and
- a few additional learnable parameters, while
- keeping the network frozen!

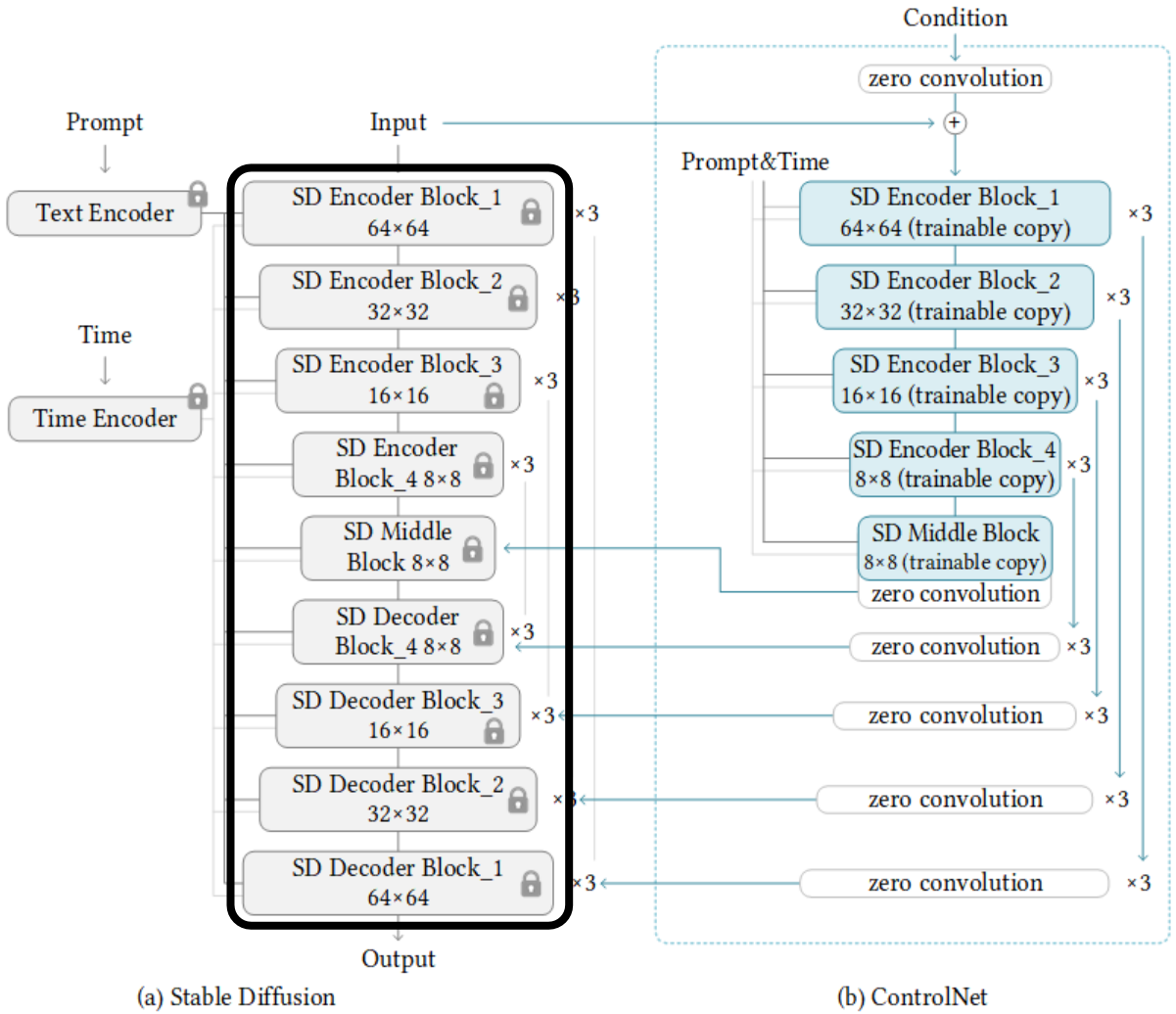
ControlNet [Zhang et al., 2023]



Key Idea:

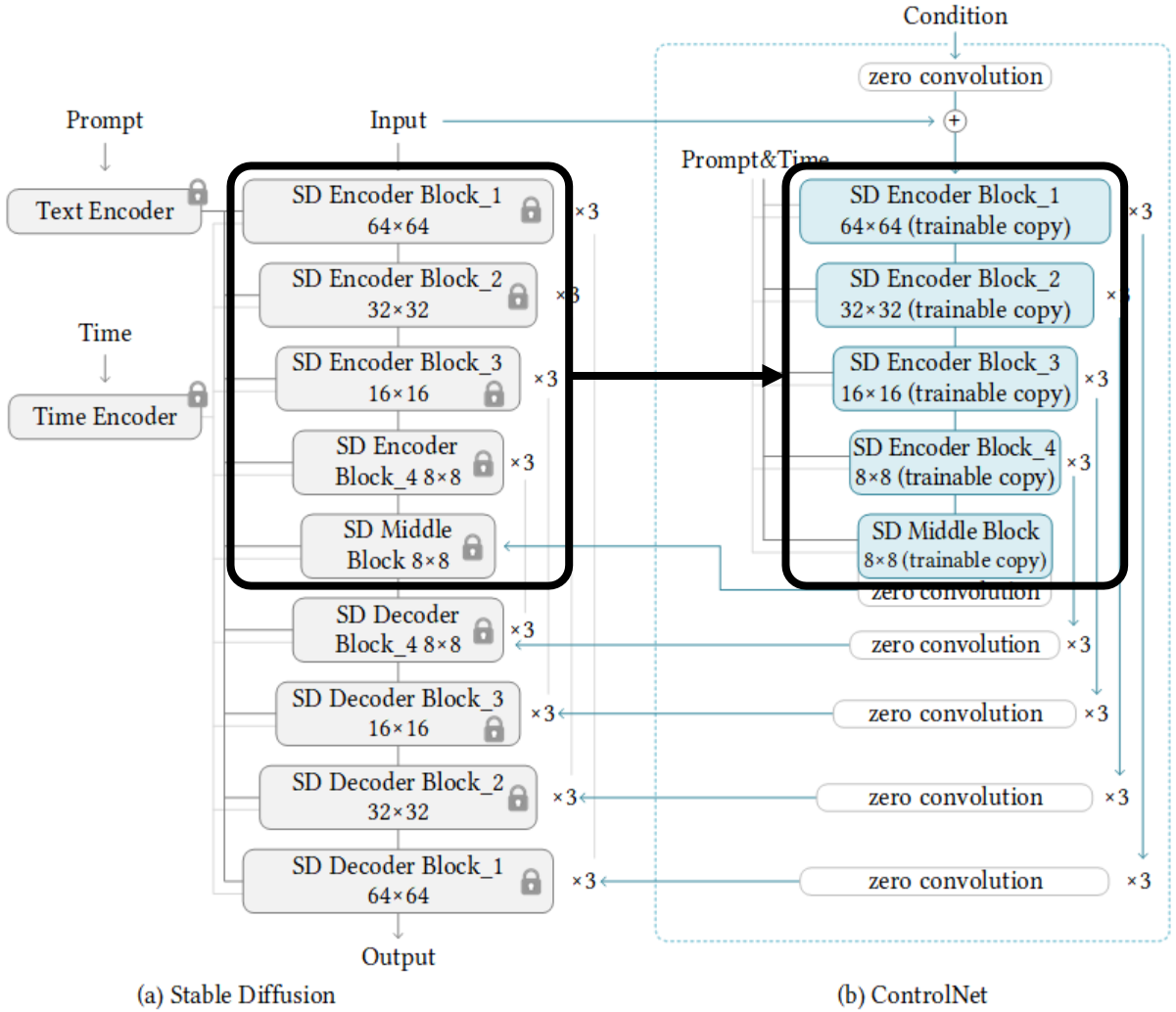
Fully leverage the pretrained noise prediction network to process the conditional image.

ControlNet [Zhang et al., 2023]



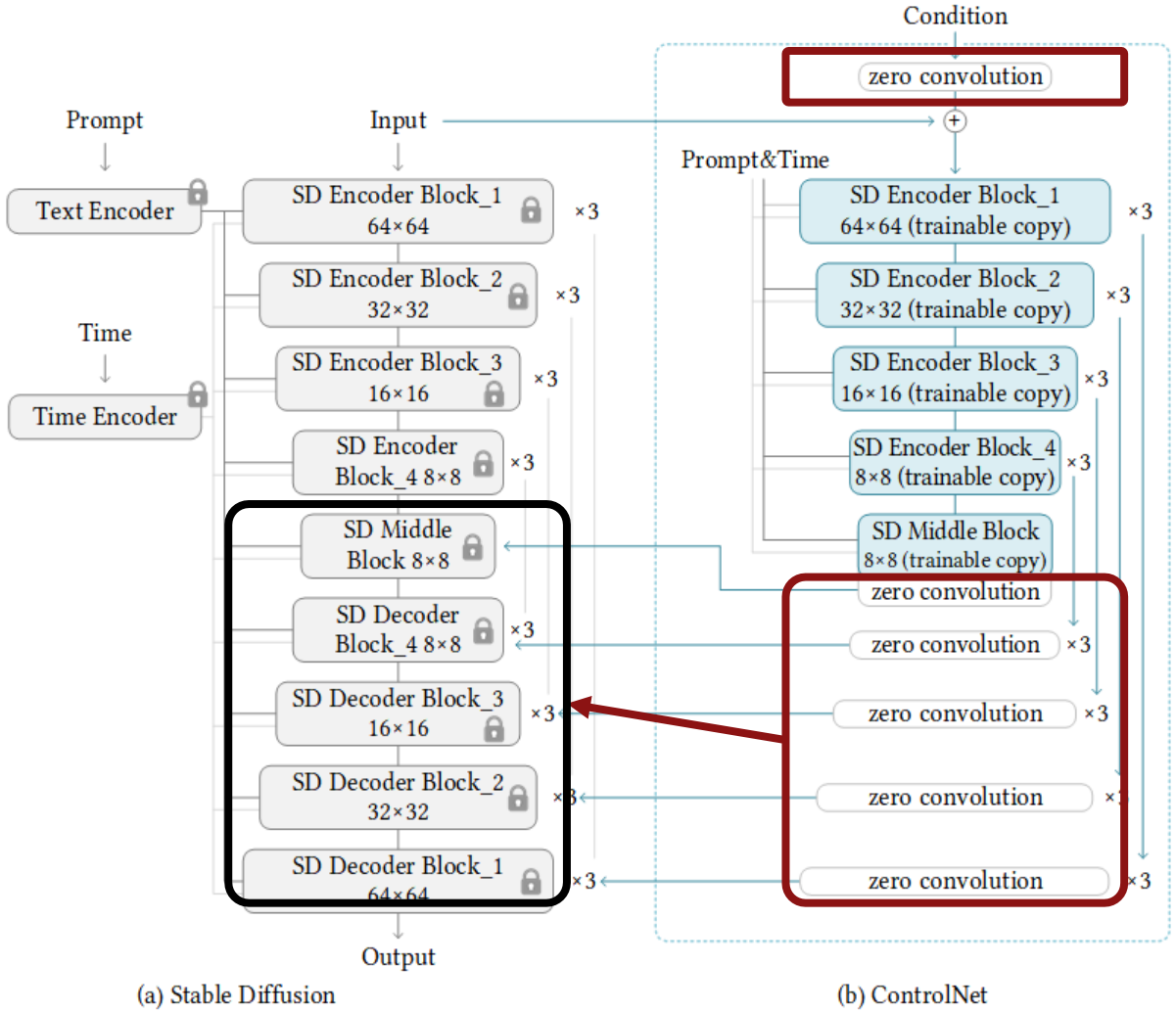
1. Freeze the noise prediction network.

ControlNet [Zhang et al., 2023]



2. For the encoding of the conditional image, copy the pretrained encoder parameters while allowing them to be updated during fine-tuning.

ControlNet [Zhang et al., 2023]



3. Combine the encoded conditional image information with the noisy image using zero convolution.

ControlNet [Zhang et al., 2023]

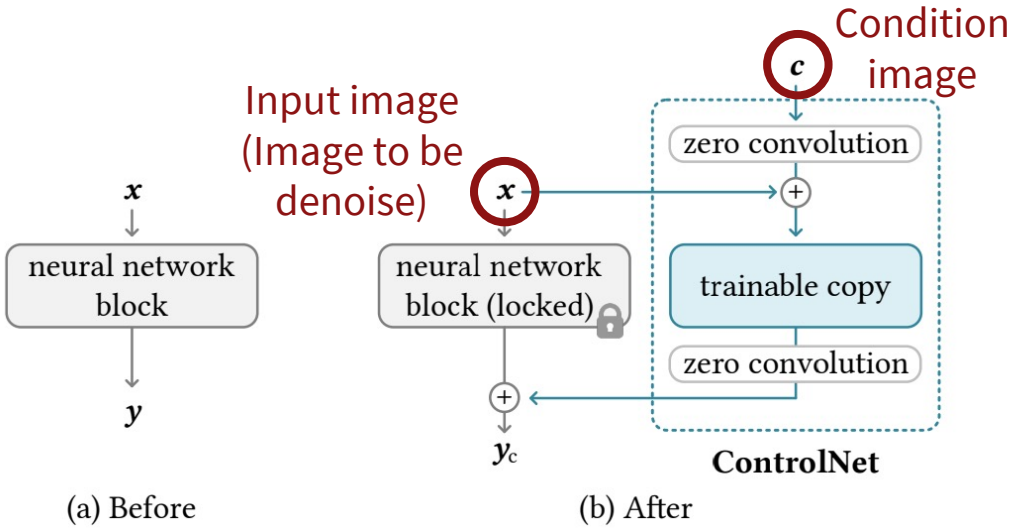
Zero Convolution Z is a 1×1 convolution layer with learnable weight (scaling) parameters a and bias (offset) parameters b, both of which are initialized with zero:

$$Z(\boldsymbol{x}; a, b) = a\boldsymbol{x} + b$$

ControlNet [Zhang et al., 2023]

ControlNet modifies each neural network block using zero convolution as follows:

$$\boldsymbol{y_c} = F(\boldsymbol{x}; \Theta) + Z(F(\boldsymbol{x} + Z(\boldsymbol{c}; a_1, b_1); \Theta_c); a_2, b_2)$$

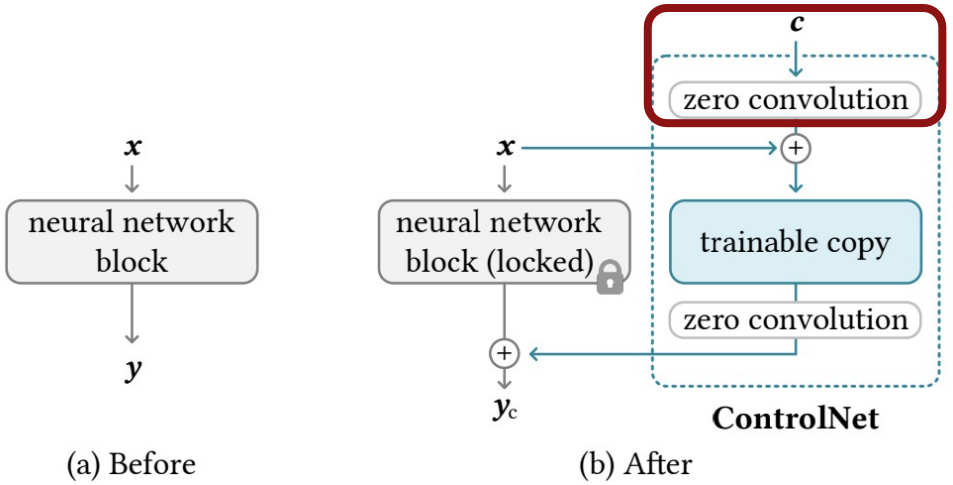


ControlNet [Zhang et al., 2023]

ControlNet modifies each neural network block using zero convolution as follows:

$$\boldsymbol{y_c} = F(\boldsymbol{x}; \Theta) + Z(F(\boldsymbol{x} + Z(\boldsymbol{c}; a_1, b_1); \Theta_c); a_2, b_2)$$

Zero in the beginning since $a_1 = b_1 = 0$.

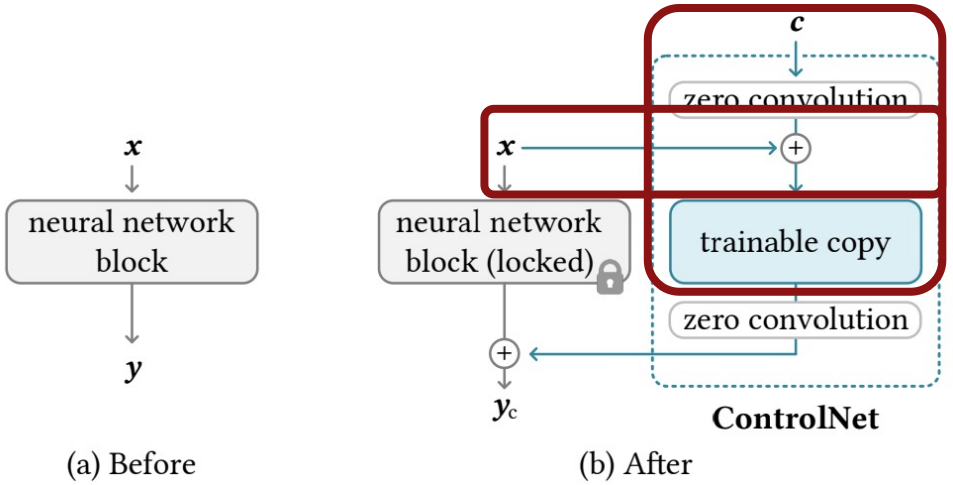


ControlNet [Zhang et al., 2023]

ControlNet modifies each neural network block using zero convolution as follows:

$$\boldsymbol{y_c} = F(\boldsymbol{x}; \Theta) + Z(F(\boldsymbol{x} + Z(\boldsymbol{c}; \boldsymbol{a_1}, \boldsymbol{b_1}); \Theta_c); \boldsymbol{a_2}, \boldsymbol{b_2})$$

Same as $F(x; \Theta)$ in the beginning since $\Theta_c = \Theta$.

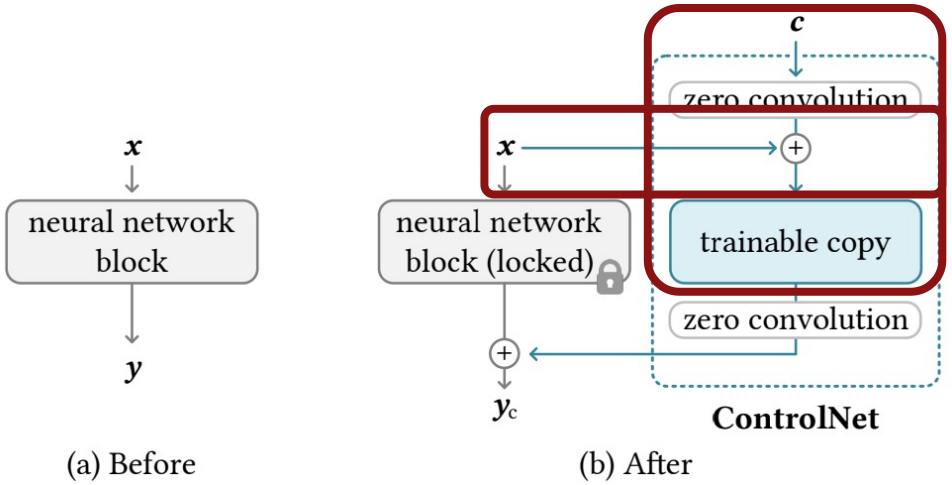


ControlNet [Zhang et al., 2023]

For encoding of the conditional image c, begin with the state where the input is x, while gradually incorporating c.

$$y_c = F(x; \Theta) + Z(F(x + Z(c; a_1, b_1); \Theta_c); a_2, b_2)$$

Same as $F(x; \Theta)$ in the beginning since $\Theta_c = \Theta$.



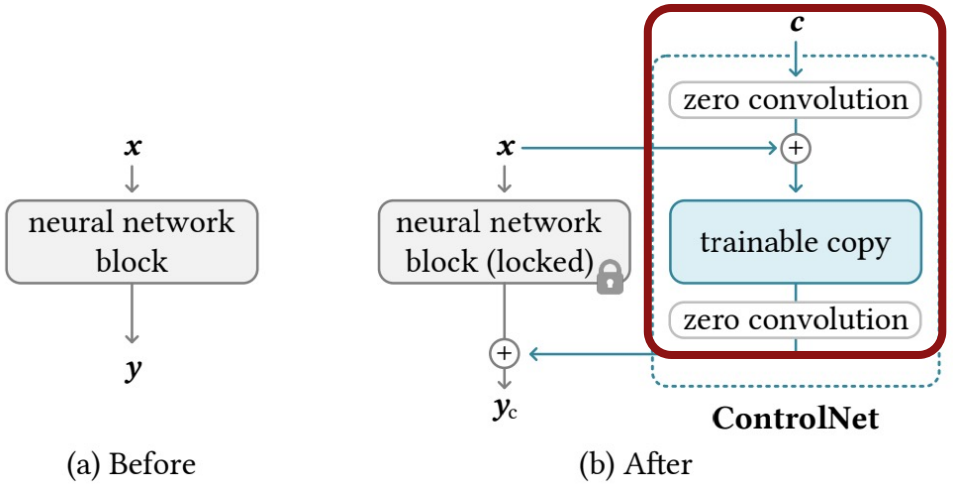
(a) Before (b) After

ControlNet [Zhang et al., 2023]

ControlNet modifies each neural network block using zero convolution as follows:

$$\boldsymbol{y_c} = F(\boldsymbol{x}; \Theta) + Z(F(\boldsymbol{x} + Z(\boldsymbol{c}; a_1, b_1); \Theta_c); a_2, b_2)$$

Zero in the beginning since $a_2 = b_2 = 0$.

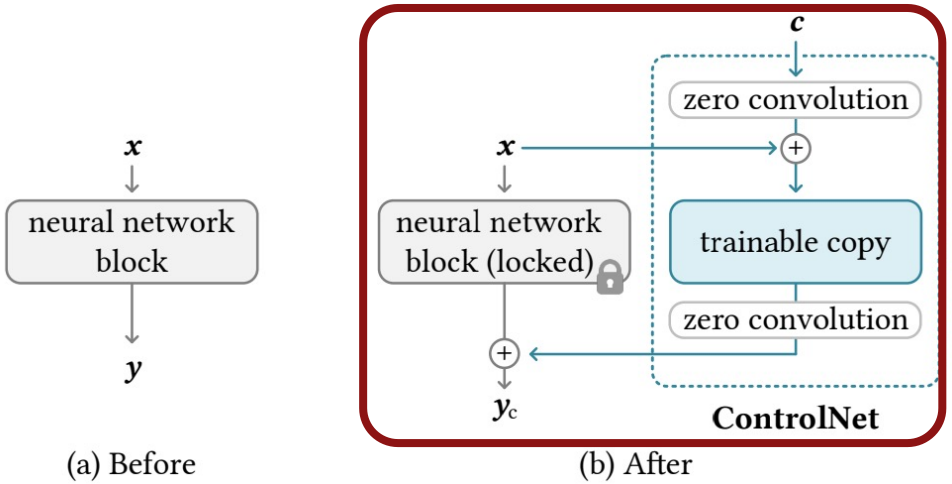


ControlNet [Zhang et al., 2023]

ControlNet modifies each neural network block using zero convolution as follows:

$$\boldsymbol{y_c} = F(\boldsymbol{x}; \Theta) + Z(F(\boldsymbol{x} + Z(\boldsymbol{c}; a_1, b_1); \Theta_c); a_2, b_2)$$

Same as $\mathbf{F}(\mathbf{x}; \boldsymbol{\Theta})$ in the beginning.

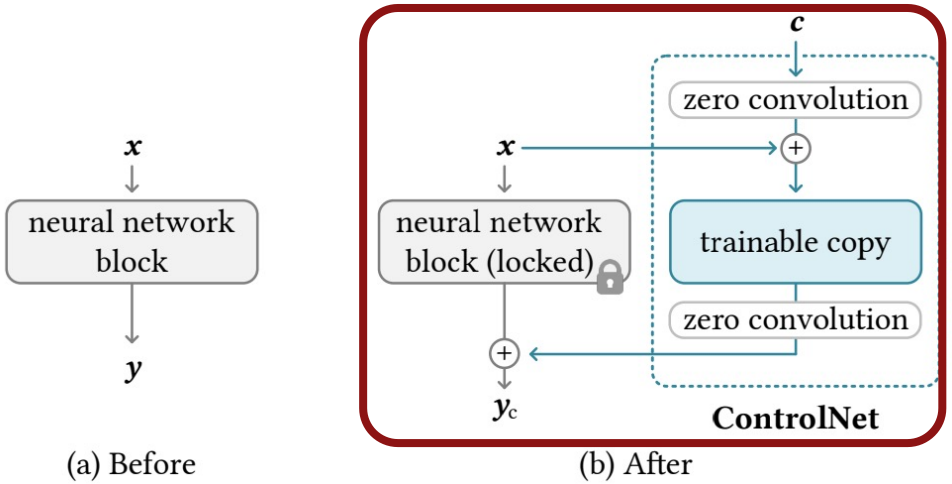


ControlNet [Zhang et al., 2023]

Also for the noise prediction, gradually incorporate the output of conditional image encoding.

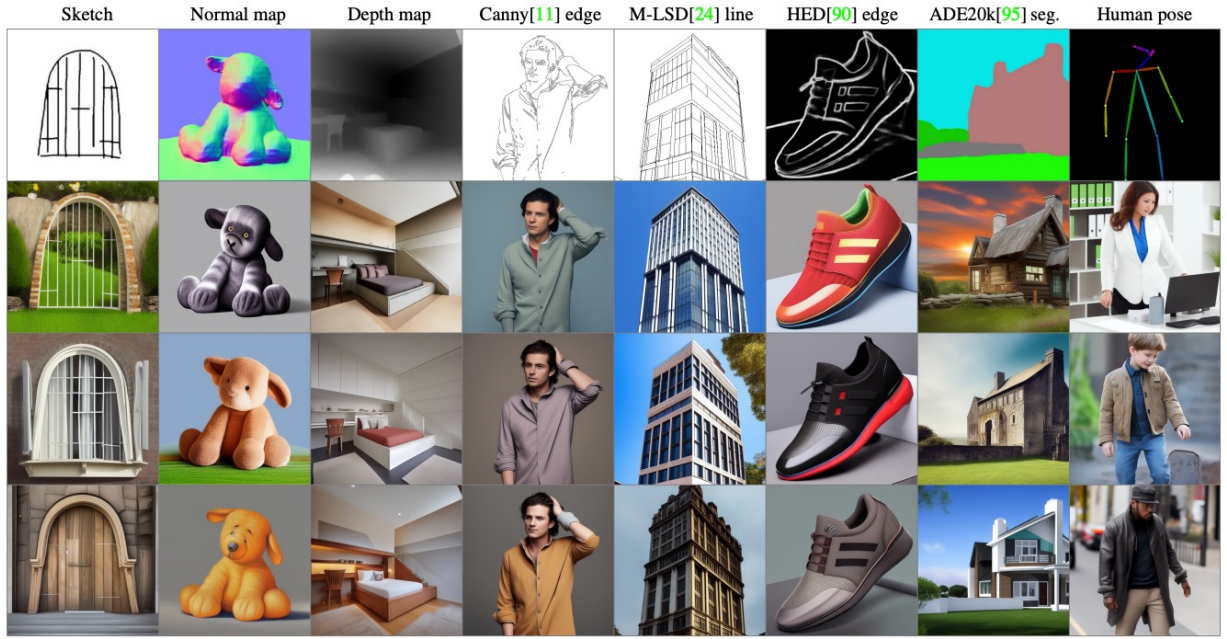
$$\boldsymbol{y_c} = F(\boldsymbol{x};\Theta) + Z(F(\boldsymbol{x} + Z(\boldsymbol{c};a_1,b_1);\Theta_c);a_2,b_2)$$

Same as $\mathbf{F(x;\Theta)}$ in the beginning.



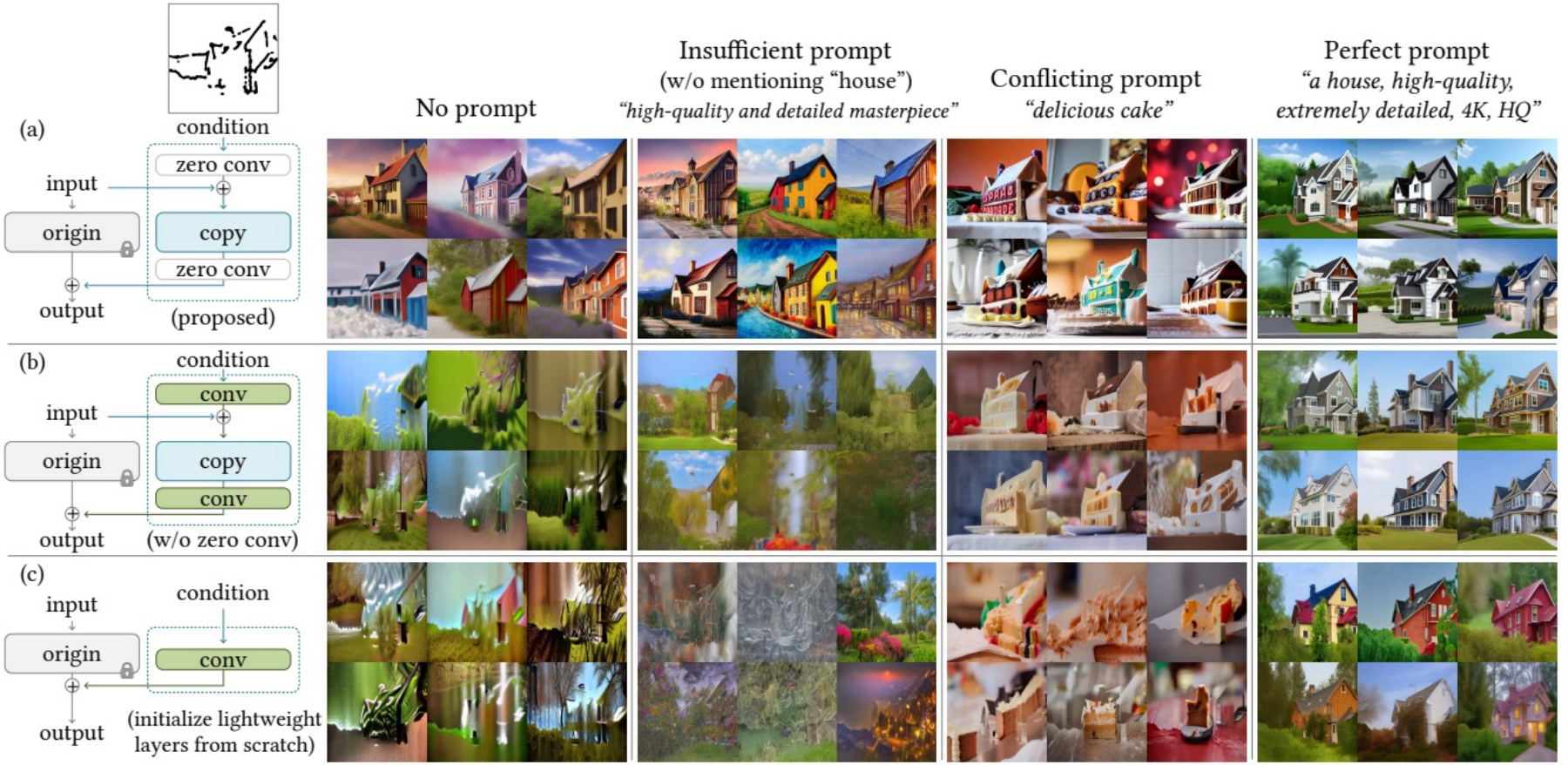
ControlNet [Zhang et al., 2023]

- The idea can be used for any image conditioning.
- The training dataset is " ~100k in size, which is 50,000 time smaller than the LAION-5B."



ControlNet [Zhang et al., 2023]

Ablative study of different architectures.



Li et al., GLIGEN: Open-Set Grounded Text-to-Image Generation, CVPR 2023.

Other Method: GLIGEN

Caption: "a photo of a hybrid between a bee and a rabbit"
Grounded text: hybrid between a bee and a rabbit, flower

Caption: "Pikachu is under a tree, digital art"
Grounded text: Pikachu, tree; Grounded style image: blue inset

Caption: "A dog / bird / helmet / backpack is on the grass"
Grounded image: red inset

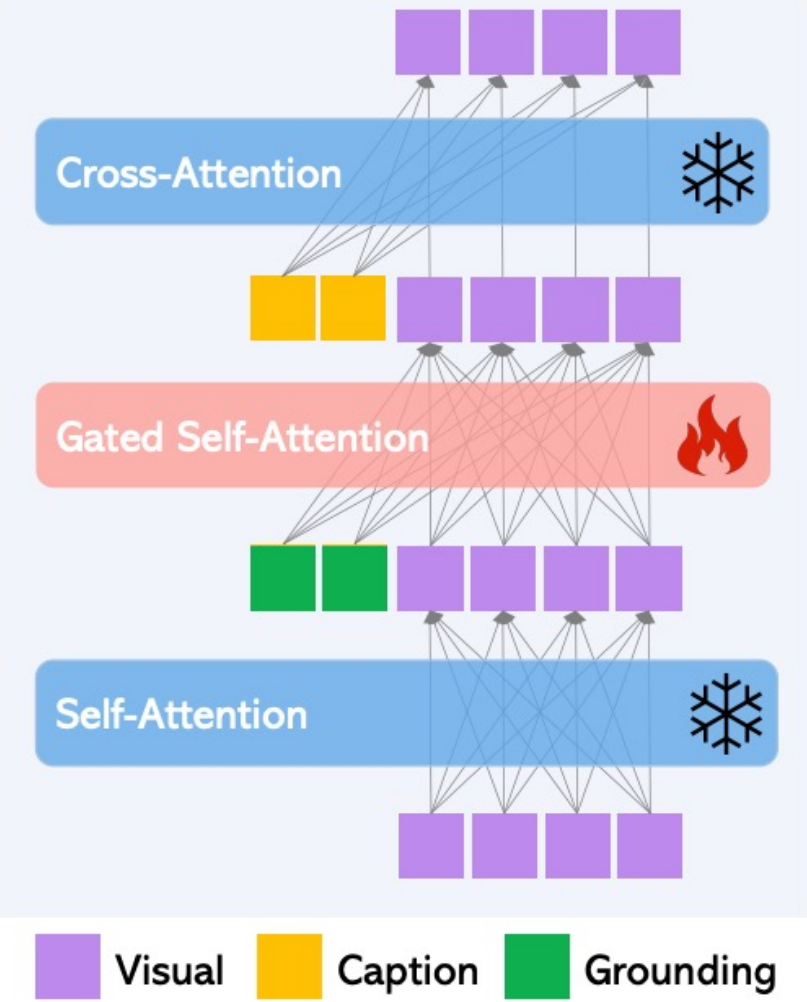
Caption: "superman / monkey / Hormer Simpson / is scratching its head"
Grounded keypoints: plotted dots on the left image

Caption: "A vibrant colorful bird sitting on tree branch"
Grounded depth map: the left image

Caption: "The beautiful scenery of a clam village near the sea"
Grounded HED map: the left image

Other Method: GLIGEN

Fine-tune the additional gated self-attention modules while freezing the network parameters.



Personalization

https://www.shruggingface.com/blog/self-portraits-with-stable-diffusion-and-lora

Personalization

LoRA

Personalization

DreamBooth

Personalization

I want to generate images of myself or my objects!

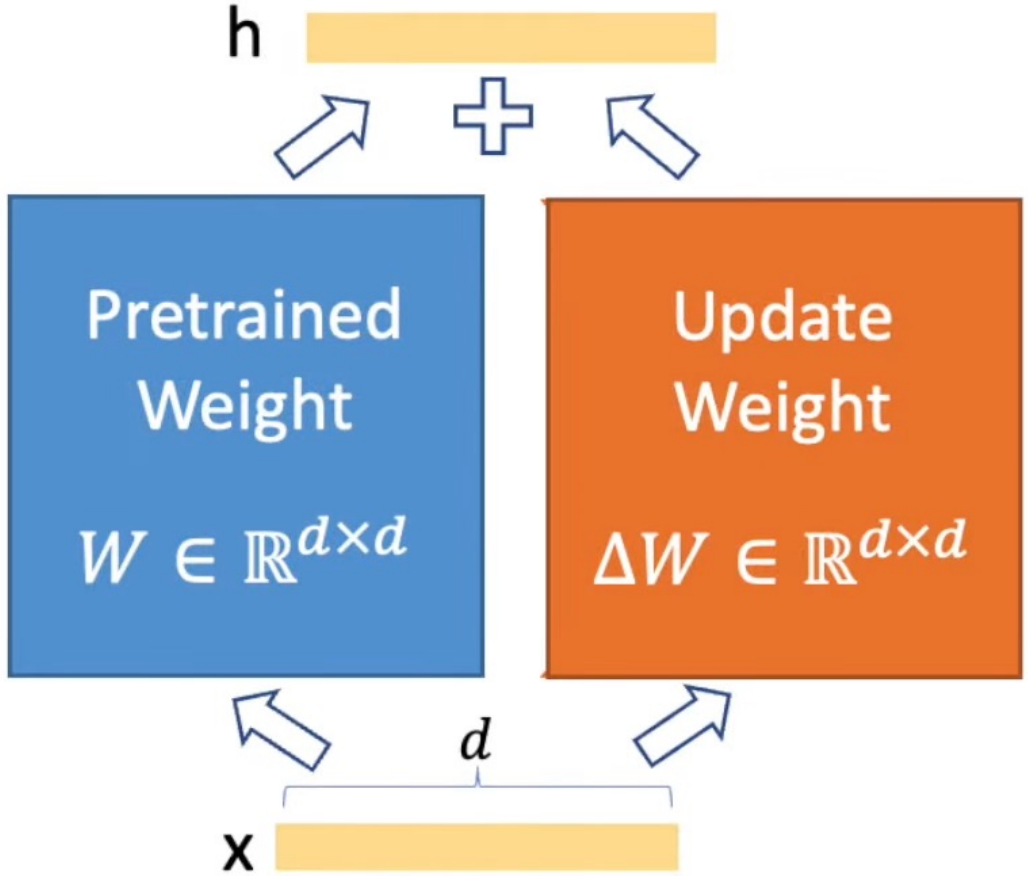
How? Use the fine-tuning approach.

Introduce a very few learnable parameters.

Hu et al., LoRA: Low-Rank Adaptation of Large Language Models.

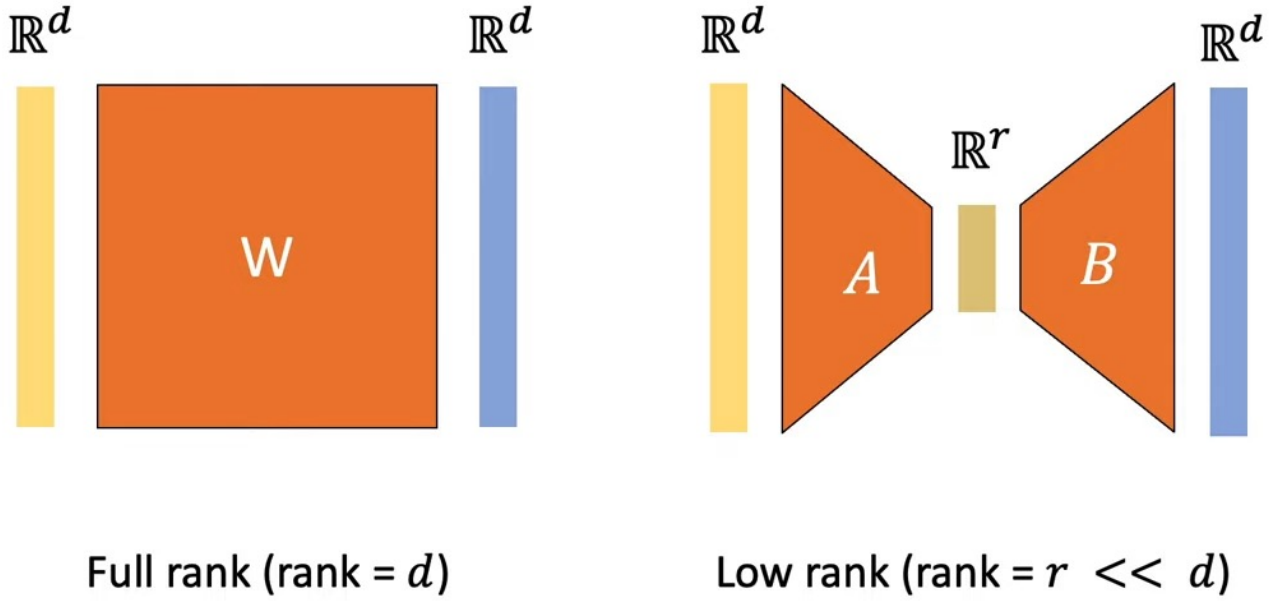
Low-Rank Adaptation

Similar to ControlNet, create an additional branch for each layer that takes the same input and outputs offsets.



Low-Rank Adaptation

- An MLP layer is the same as multiplying a matrix and applying an activation function.
- Approximate the matrix as a multiplication of two low-rank matrices.



Low-Rank Adaptation

Low-Rank Approximation:

Equivalent to having two MLP layers with a low-dimensional intermediate output.